# Logos 系列 FPGA 算数处理模块(APM)

# 用户指南

(UG020003, V1.4)

(2022.8.13)

**深圳市紫光同创电子有限公司**

**版权所有　侵权必究**

# 文档版本修订记录

| 版本号 | 发布日期 | 修订记录 |
|---|---|---|
| V1.0 | 2018/04/03 | 1. 初始版本 |
| V1.1 | 2018/12/18 | 1. 封面新增最新的版本更新时间；页脚删除版本信息 |
| V1.2 | 2021/02/23 | 1. 更新文档格式<br>2. 更新文档名称与其他 UG 统一<br>3. 修正部分笔误 |
| V1.3 | 2021/03/26 | 1. 更新图 2-1 APM 整体功能框图，及各工作模式示意图<br>2. 新增部分功能特性描述<br>3. 更新 2.2.2 端口说明、2.2.3 参数说明<br>4. 更正 2.3APM 工作模式中描述，时序说明<br>5. 新增 3.2 设计建议的说明，3 附录新增 GTP 例化参考 |
| V1.4 | 2022/08/13 | 1. 更新 1 总体介绍说明，更正支持的乘积运算说明<br>2. 新增 2.3 APM 工作模式中取负描述，3.5 使用 GTP 例化为乘累加模式取负说明<br>3. 更新 2.1.2 预加单元描述<br>4. 更新 3.1 级联可实现功能级联端口描述<br>5. 新增 2.3.1 乘法模式参数说明 |

# 目 录

**文档版本修订记录 ..... 1**

**目 录 ..... 2**

**表目录 ..... 4**

**图目录 ..... 5**

**名词术语解释 ..... 6**

**1 总体介绍 ..... 7**

**2 功能描述 ..... 8**

2.1 APM 结构框图 ..... 8

2.1.1 I/O 单元 ..... 9

2.1.2 预加单元 ..... 9

2.1.3 乘法单元 ..... 9

2.1.4 累加单元 ..... 9

2.2 GTP 说明 ..... 10

2.2.1 GTP 结构框图 ..... 10

2.2.2 端口说明 ..... 10

2.2.3 参数说明 ..... 11

2.2.4 模式说明 ..... 12

2.3 APM 工作模式 ..... 14

2.3.1 乘法模式 ..... 14

2.3.2 通用乘加模式 ..... 16

2.3.3 乘累加模式 ..... 17

2.3.4 FIR 模式 ..... 19

**3 附录 ..... 21**

3.1 级联可实现功能 ..... 21

3.2 设计建议 ..... 22

3.3 使用 GTP 例化为乘法模式 ..... 22

3.4 使用 GTP 例化为乘加模式 ..... 24

3.5 使用 GTP 例化为乘累加模式 ..... 25

**免责声明 ..... 28**

# 表目录

表 2-1 GTP_APM_E1 端口列表 ..... 10
表 2-2 GTP_APM_E1 参数列表 ..... 11
表 2-3 MODE 端口功能说明 ..... 13
表 2-4 MODEZ[2:1]功能说明 ..... 13
表 2-5 典型 FIR 滤波器实现 ..... 19
表 3-1 APM 级联可实现功能描述 ..... 21

# 图目录

图 2-1 APM 整体功能框图 ..... 8
图 2-2 X, XB 及 Pre-adder 功能框图 ..... 8
图 2-3 GTP_APM_E1 单元 ..... 10
图 2-4 乘法模式应用示意图 ..... 14
图 2-5 带预加的乘法模式应用示意图 ..... 15
图 2-6 乘法模式的典型时序图 ..... 15
图 2-7 通用乘加模式应用示意图 ..... 16
图 2-8 通用乘加模式的典型时序 ..... 17
图 2-9 乘累加模式应用示意图 ..... 17
图 2-10 带预加的乘累加应用示意图 ..... 18
图 2-11 乘累加模式的典型时序 ..... 18
图 2-12 Systolic FIR 功能级联示意图 ..... 19

# 名词术语解释

| Abbreviations 缩略语 | Full Spelling 英文全拼 | Chinese Explanation 中文解释 |
|---|---|---|
| GTP | Generic Technology Primitive | 通用技术原语 |
| APM | Arithmetic Process Module | 算术处理单元 |
| SRB | Signal Relay Block | 信号互连模块 |

# 1 总体介绍

Logos 系列产品提供了 APM（Arithmetic Process Module，算术处理单元）模块，为 Logos 产品提供了高效的数字信号处理能力。

APM 在 Logos 产品中按列分布，其主要特性有：

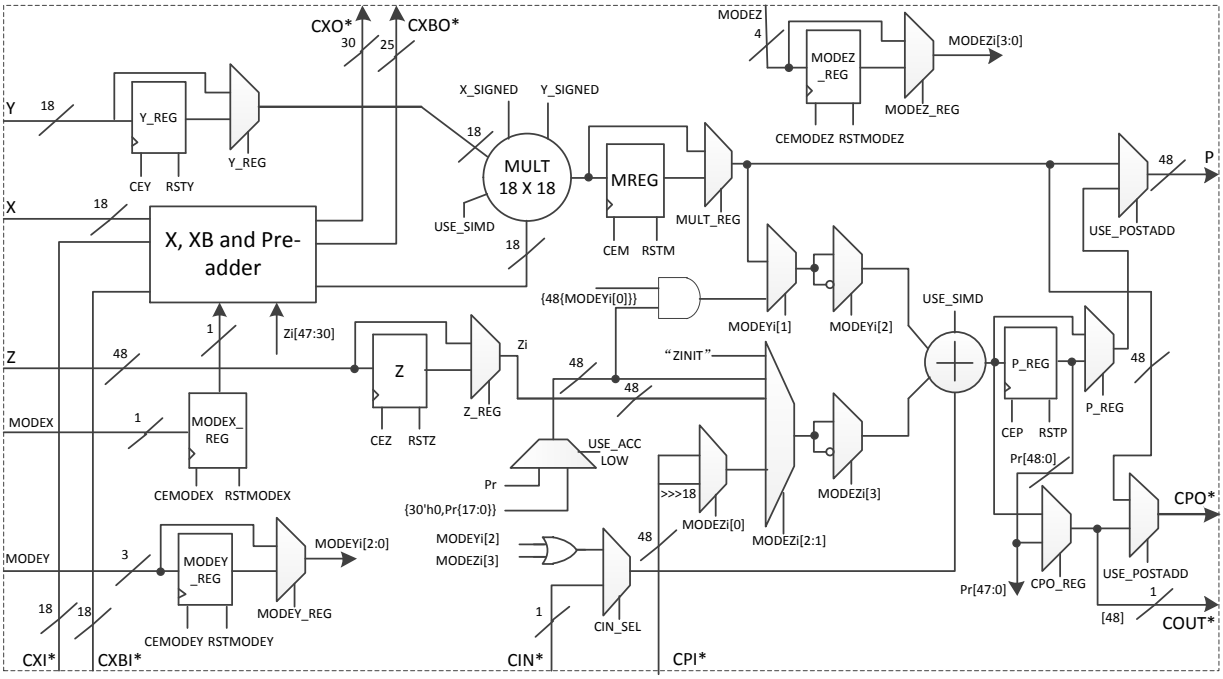
- 单个 APM 支持 1 个 18×18 或 2 个 9×9 乘积运算
- 可灵活配置，支持乘、乘累加以及通用乘加等模式
- 可选的 Preadd（预加）功能
- 可选的输入、输出及两级内部流水寄存器
- 支持部分输入源和操作符的动态选择，可以通过对 APM 级联获得更高位宽运算
- 支持 48bit Postadd（累加）功能

APM的实现可通过深圳市紫光同创电子有限公司的软件Pango Design Suite（后文简称为PDS）来完成。为方便用户使用，Pango Design Suite内嵌的IP Compiler工具提供IP的生成，根据配置生成各种模式的APM，详见“Logos系列Module IP用户指南”之APM。

# 2 功能描述

## 2.1 APM 结构框图

APM 主要由 I/O Unit（I/O 单元）、Preadd Unit（预加单元）、Mult Unit（乘法单元）和 Postadd Unit（累加单元）四个功能单元组成，整个 APM 逻辑结构如下图所示。




注：*这些信号为级联信号

图 2-1 APM 整体功能框图

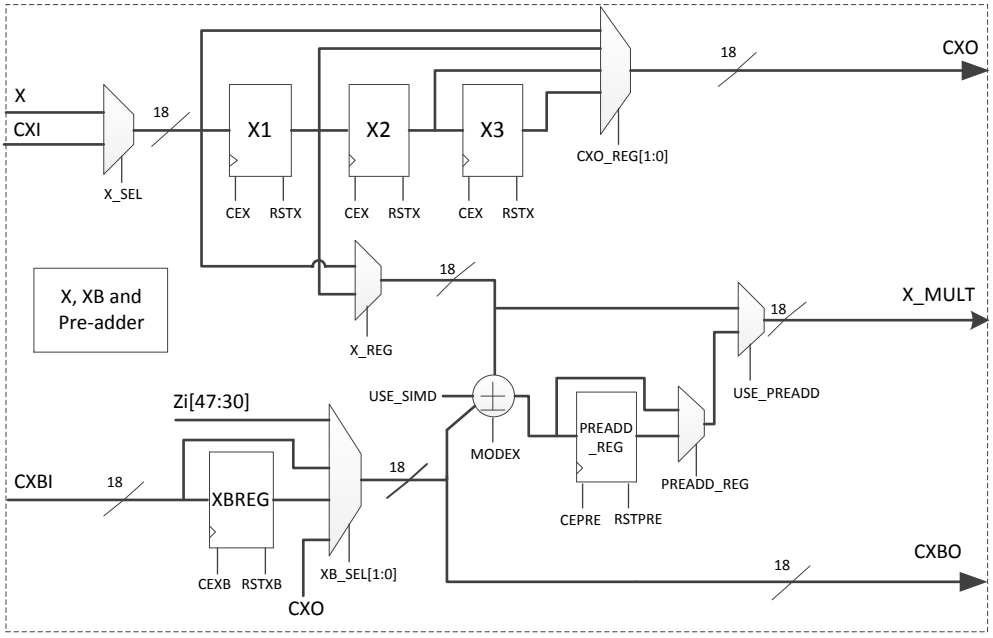



图 2-2 X, XB 及 Pre-adder 功能框图

### 2.1.1 I/O 单元

- 实现数据输入/输出寄存。
- 各个寄存器的 CE 信号（高有效）和 RST 信号（高有效）都有其各自独立的 CE 和 RST 输入来控制。RST 异步/同步性质由共享参数 ASYNC 决定。
- 模式端口按逻辑归属分成三组 MODEX，MODEY[2:0]，MODEZ[3:0]。每组模式端口控制由各自的独立的 REG，RST，CE 参数控制。
- 实现 APM 正向级联(CPO=>CPI，CXO=>CXI)和反向级联(CXBO=>CXBI)。APM 级联用于实现滤波器以及高位宽乘法器如 36*36。
- 实现对常数输入和重复的符号位简化接线。

### 2.1.2 预加单元

- 硬件实现为 18±18，输出 18 位结果；或两个 9±9，各输出 9 位结果(当 SIMD=1)；由于输入、输出位宽相同，用户需考虑溢出的情况。
- 当 XSIGNED=0 时，Preadder 输出结果为无符号数；XSIGNED=1 时 Preadder 结果判定为有符号数。
- Preadder 可被旁路（PREADD=0）。

### 2.1.3 乘法单元

- 实现一个 18×18，结果带符号扩展为 48 位；或两个 9×9（当 SIMD=1），9×9 结果带符号扩展为 24 位。
- X/Y 输入操作数的符号性(signedness)由参数 XSIGNED/YSIGNED 决定，该参数为两个 9*9 乘法器共享。

### 2.1.4 累加单元

- 实现一个 48 位加减法/累加器；或两个 24 位加减法/累加器（当 SIMD=1），两者共享一个 carryin（进位）。
- 累加单元为有符号数加法。
- Postadder 的两个主要输入端为乘法器和 Zmux，两者之一可任选取负值。
- 为减少 APM 端口数，节约布线资源，Z 端口高 18 位也可以为 Preadder 所复用，原则上 Postadder 此时不应再从 Z 端口取操作数。
- Postadder 可被旁路（当 POSTADD=0），此时 APM 输出乘法器结果。

## 2.2 GTP 说明

GTP_APM_E1 单元集成了 APM 的各种工作模式，用户可通过对接口和参数的配置，实现各种模式的选择和 APM 功能实现。

### 2.2.1 GTP 结构框图

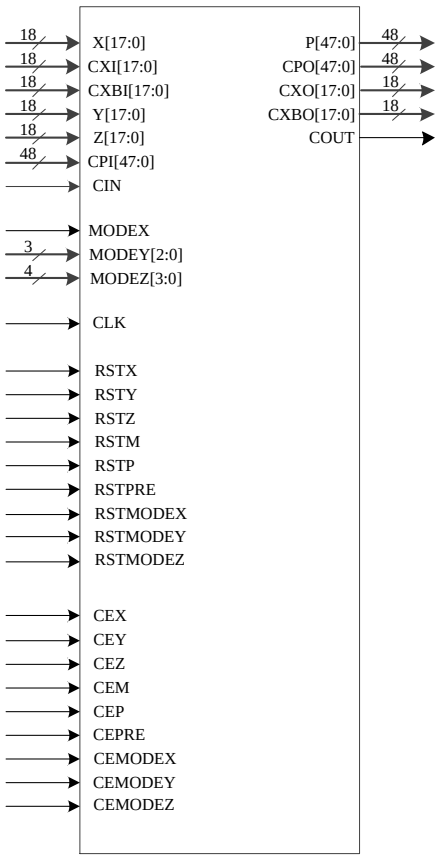



图 2-3 GTP_APM_E1 单元

### 2.2.2 端口说明

表 2-1 GTP_APM_E1 端口列表

| 端口名 | I/O | 描述 |
|---|---|---|
| X[17:0] | I | 并行数据输入 X，X[17:0]作为乘法器或预加器的 X 输入 |
| CXI[17:0] | I | 级联 X 输入，来自上一个 APM 模块的 CXO 端口 |
| CXBI[17:0] | I | 级联 XB 输入，来自上一个 APM 模块的 CXBO 端口 |
| Y[17:0] | I | 并行数据输入 Y，乘法器的 Y 输入 |
| Z[47:0] | I | 并行数据输入 Z，Z[47:0]为累加器输入数据，Z[47:30]可作为预加器输入 |
| CPI[47:0] | I | 级联 P 输入，来自上一个 APM 模块的 CPO 端口 |
| CIN | I | 级联 CIN 输入，来自上一个 APM 模块的 COUT 端口 |
| MODEX | I | APM 动态 X 端控制操作符，具体作用见模式说明 |
| MODEY[2:0] | I | APM 动态 Y 端控制操作符，具体作用见模式说明 |
| MODEZ[3:0] | I | APM 动态 Z 端控制操作符，具体作用见模式说明 |

| 端口名 | I/O | 描述 |
|---|---|---|
| CLK | I | 时钟输入，用于所有内部寄存器 |
| CEX | I | X 端口输入寄存器的时钟使能信号，高有效，具体使用几个寄存器由参数 X_REG、CXO_REG 控制 |
| RSTX | I | 高有效的 X 复位信号，复位所有 X 输入寄存器 |
| CEY | I | YREG 的时钟使能信号，在 Y_REG=1 时使用 |
| RSTY | I | 高有效的 Y 复位信号，复位 Y 输入寄存器 |
| CEZ | I | ZREG 的时钟使能信号，在 Z_REG=1 时使用 |
| RSTZ | I | 高有效的 Z 复位信号，复位 Z 输入寄存器 |
| CEPRE | I | PREREG 的时钟使能信号 |
| RSTPRE | I | 高有效的 PRE 复位信号，复位 PRE 寄存器 |
| CEM | I | MREG 的时钟使能信号 |
| RSTM | I | 高有效的 M 复位信号，复位 M 寄存器 |
| CEP | I | PREG 的时钟使能信号 |
| RSTP | I | 高有效的 P 复位信号，复位 P 寄存器 |
| CEMODEX | I | MODEXREG 的时钟使能信号 |
| RSTMODEX | I | 高有效的 MODEX 复位信号，复位 MODEX 输入寄存器 |
| CEMODEY | I | MODEYREG 的时钟使能信号 |
| RSTMODEY | I | 高有效的 MODEY 复位信号，复位 MODEY 输入寄存器 |
| CEMODEZ | I | MODEZREG 的时钟使能信号 |
| RSTMODEZ | I | 高有效的 MODEZ 复位信号，复位 MODEZ 输入寄存器 |
| P[47:0] | O | APM 的 48 位并行数据输出 |
| CPO[47:0] | O | 级联 P 输出，连接下一个 APM 的 CPI |
| COUT | O | 级联 CIN 输出，连接下一个 APM 的 CIN |
| CXO[17:0] | O | 级联 X 输出，连接下一个 APM 的 CXI |
| CXBO[17:0] | O | 级联 XB 输出，连接下一个 APM 的 CXBI |

### 2.2.3 参数说明

表 2-2 GTP_APM_E1 参数列表

| 参数名 | 设置值 | 描述 |
|---|---|---|
| GRS_EN | FALSE：不使能<br>TRUE：使能 | 全局复位信号使能 |
| ASYNC_RST | 0：同步复位<br>1：异步复位 | 复位选择 |
| X_SIGNED | 0：无符号<br>1：有符号 | X 的符号性 |
| Y_SIGNED | 0：无符号<br>1：有符号 | Y 的符号性 |
| USE_PREADD | 0：不使用<br>1：使用 | preadder 选择 |
| USE_POSTADD | 0：不使用<br>1：使用 | postadder 选择 |

| 参数名 | 设置值 | 描述 |
|---|---|---|
| X_REG | 0：不使用<br>1：使用 | X Register 选择 |
| CXO_REG | 2’b00：0 个周期<br>2’b01：1 个周期<br>2’b10：2 个周期<br>2’b11：3 个周期 | X 输出寄存器延迟选择 |
| Y_REG | 0：不使用<br>1：使用 | Y 寄存器选择 |
| Z_REG | 0：不使用<br>1：使用 | Z 寄存器选择 |
| PREADD_REG | 0：不使用<br>1：使用 | preadder 寄存器选择 |
| P_REG | 0：不使用<br>1：使用 | postadder 寄存器选择 |
| MULT_REG | 0：不使用<br>1：使用 | multiplier 寄存器选择 |
| MODEX_REG | 0：不使用<br>1：使用 | MODEX 寄存器选择 |
| MODEY_REG | 0：不使用<br>1：使用 | MODEY 寄存器选择 |
| MODEZ_REG | 0：不使用<br>1：使用 | MODEZ 寄存器选择 |
| CPO_REG | 0：不使用<br>1：使用 | COUT、CPO 寄存器选择 |
| X_SEL | 0：X<br>1：XI | X 输入选择 |
| XB_SEL | 2’b00：Zi[47:30]<br>2’b01：CXBI<br>2’b10：CXBI_reg<br>2’b11：CXO | 级联 preadder 输入选择 |
| Z_INIT | 用户自定义 | Z 静态输入 |
| USE_ACCLOW | 0：不使用<br>1：使用 | 累加器仅使用低 18 位反馈输入选择使能 |
| USE_SIMD | 0：模式 18<br>1：模式 9 | SIMD 模式选择 |
| CIN_SEL | 0：(MODEZ[3]\|MODEY[2])<br>1：CIN | postadder 进位输入选择 |

### 2.2.4 模式说明

APM 包含一个 18bit 预加器，一个 18bit*18bit 的二进制补码乘法器，其中的数据通路选择由 MODEX、MODEY、MODEZ 控制，之后连接到累加器单元。APM 的输入连接到各算数单元，X、CXBI、Y、Z 输入都可以手动选择寄存 1 次以满足不同 APM 应用。控制信

号输入也可以选择进行一次寄存。要达到 Data Sheet 最大速度需要使能流水寄存器。

控制 APM 工作模式的为 MODEX，MODEY[2:0]，MODEZ[3:0]三组信号，它们的端口说明如下表所示：

表 2-3 MODE 端口功能说明

| 信号 | 说明 |
|---|---|
| MODEX | preadder 加减选择<br>0：加<br>1：减 |
| MODEZ[3] | 对 Zmux 输出取反，高有效 |
| MODEZ[2:1] | Zmux 输入选择 |
| MODEZ[0] | 对上一级级联输出算术右移 18 位，高有效，用于实现大位宽乘法器如 36*36 |
| MODEY[2] | 对 Ymux 输出取反，高有效 |
| MODEY[1] | 乘法器输出到 Postadder，低有效 |
| MODEY[0] | 0：短路乘法器输出<br>1：使用累加器反馈 |

Zmux 输入有多种选择，具体由 MODEZ[2:1]决定：

表 2-4 MODEZ[2:1]功能说明

| MODEZ[2:1] | 说明 |
|---|---|
| 00 | “ZINIT”[47:0] 累加器初始化常数 |
| 01 | 累加器反馈 |
| 10 | Z 输入端 |
| 11 | 上一级 APM 结果级联输出 |

## 2.3 APM 工作模式

### 2.3.1 乘法模式

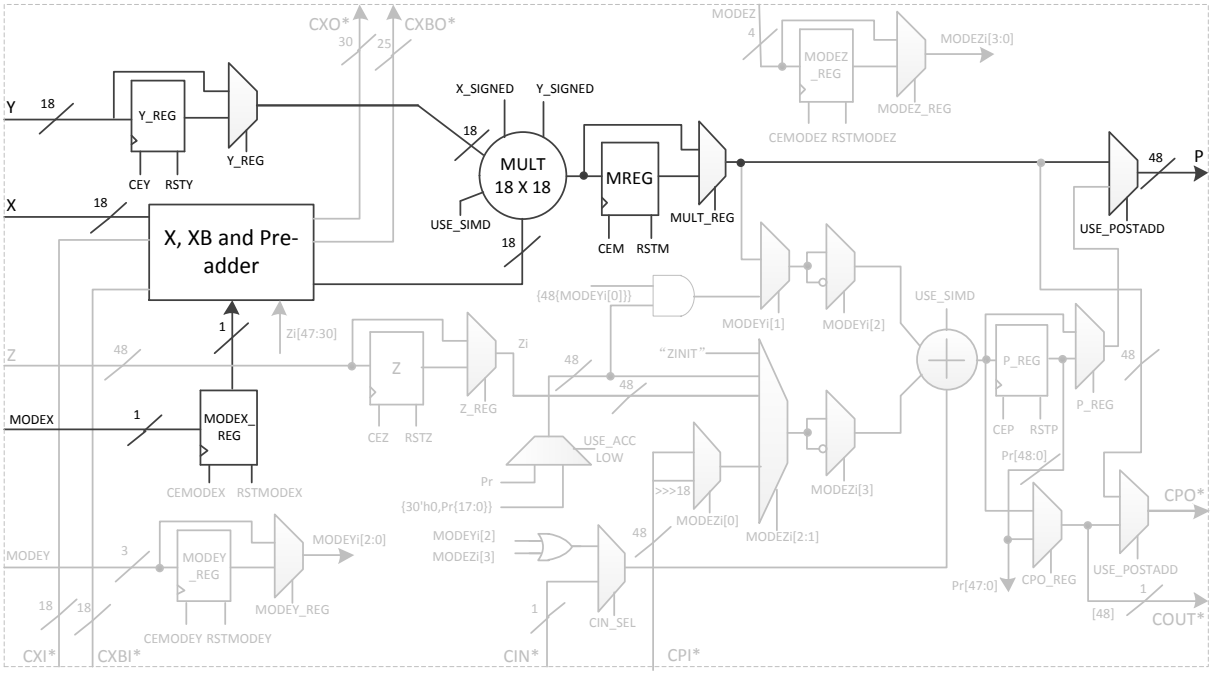



图 2-4 乘法模式应用示意图

如上图所示，APM配置成乘法模式时，其等效算术表达式为：

$$P=X\times Y$$

APM配置成乘法模式时主要特性有：

- 每个APM可实现两个9×9运算（USE_SIMD=1）或一个18×18运算（USE_SIMD=0）
- 支持有符号数和无符号数
- 输入/输出寄存器可选

使能APM中的Preadd Unit后，APM可配置成带预加的乘法模式，其算术表达式为：

$$P=Y\times(X\pm Z[47:30])$$

该模式下，每个APM可实现两个9×(9±9)运算，或1个18×(18±18)运算。除了可选的输入、输出寄存器，带预加的乘法模式还可选择使能内部流水寄存器，其应用示意图如下图所示：

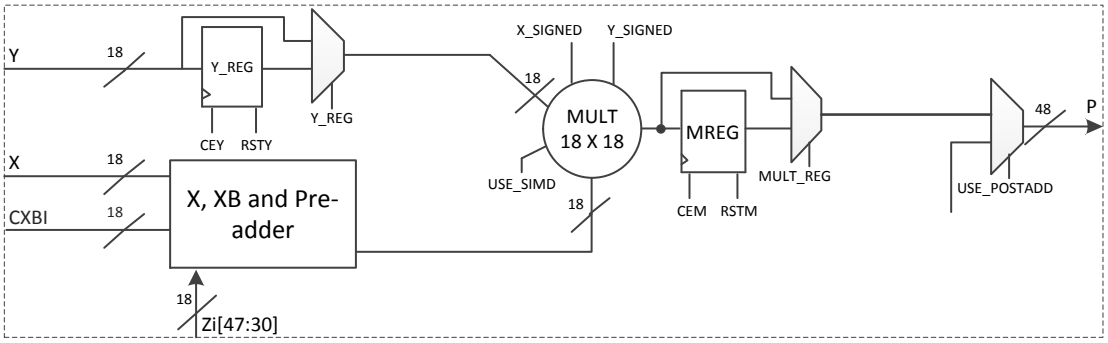



图 2-5 带预加的乘法模式应用示意图

乘法模式可以使用Logos Simple Multiplier IP进行例化（参见“Logos系列Module IP用户指南”之APM），也可以使用GTP_APM_E1单元进行例化。附录中包含使用GTP例化为乘法模式的示例，供用户在使用时进行参考配置。

乘法模式的典型时序如下，其中P=Y×(X+Z[47:30])，使能MULT_REG后，P端对应X,Y端输入数据的计算结果在下一个时钟上升沿输出。

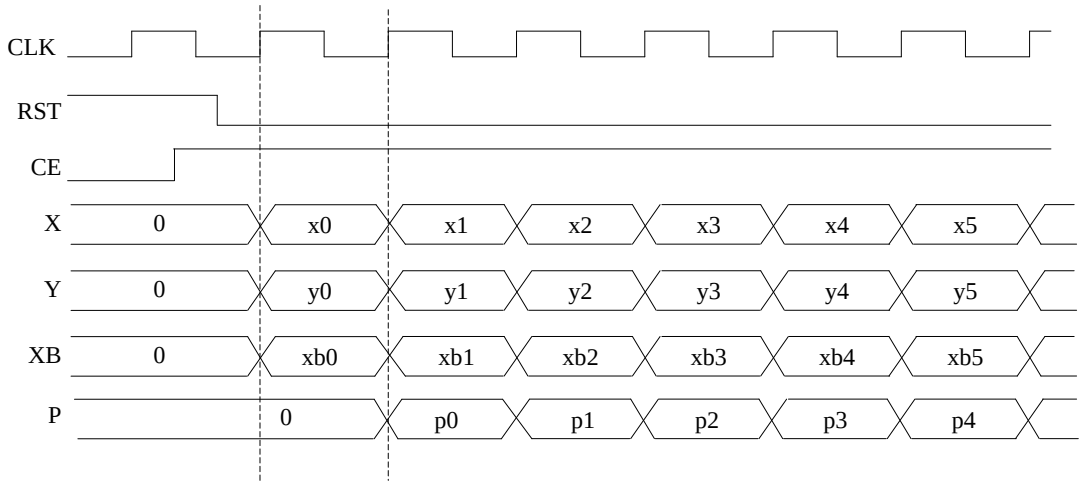



图 2-6 乘法模式的典型时序图

### 2.3.2 通用乘加模式

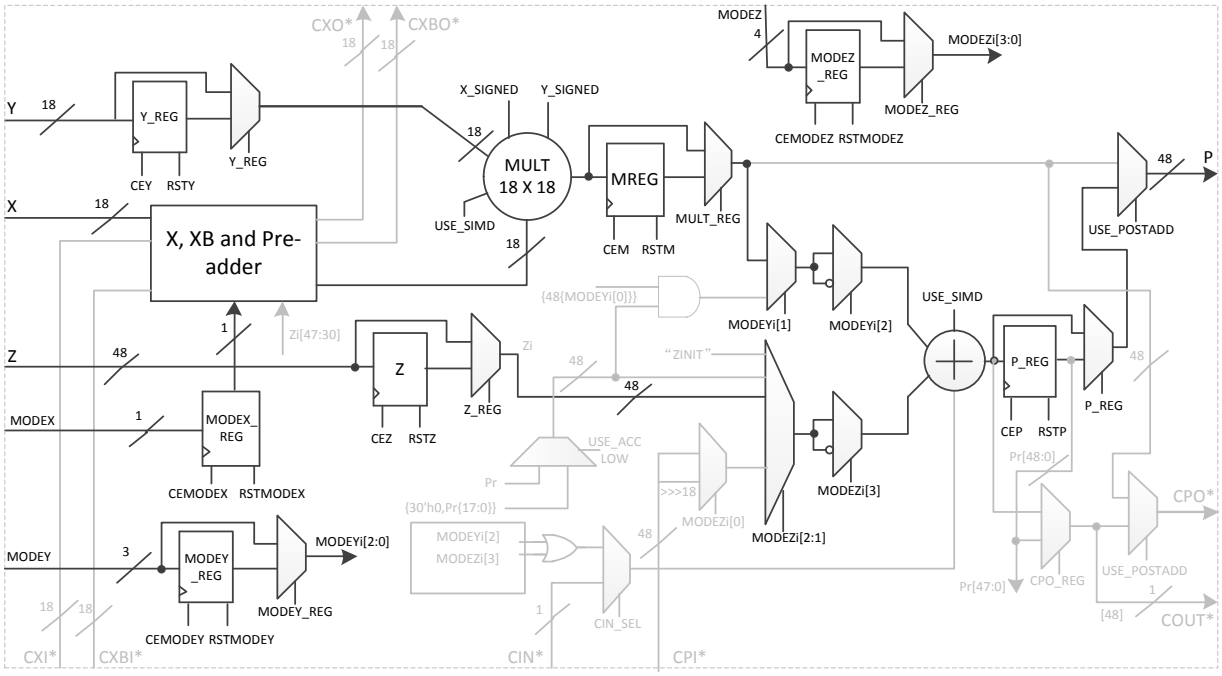



图 2-7 通用乘加模式应用示意图

如上图所示，APM配置成通用乘加模式时，其算术表达式为：

$$P=X\times Y\pm Z$$

通用乘加模式的主要特性有：

- 每个APM可实现两个9×9±24运算或一个18×18±48的乘加运算
- 支持有符号数和无符号数
- 可选的输入/输出寄存器

当MODEY[2]==1，乘法器输出取反（在Ymux处取反）后，在累加器处加1，等价于Ymux取负（此处即乘法器输出取负）。

当MODEZ[3]==1，Zmux输出取反后，在累加器处加1，等价于Zmux取负。

注意MODEY[2]与MODEZ[3]不能同时为1，可根据需要配置。

通用乘加模式可以使用Logos Multiply-Adder IP进行例化（参见“Logos系列Module IP用户指南”之APM），也可以使用GTP_APM_E1单元进行例化。附录中包含使用GTP例化为乘加模式的示例，供用户在使用时进行参考配置。通用乘加模式的典型时序如下，其中P=X×Y+Z，使能P_REG后，P端对应X,Y端输入数据的计算结果在下一个时钟上升沿输出。

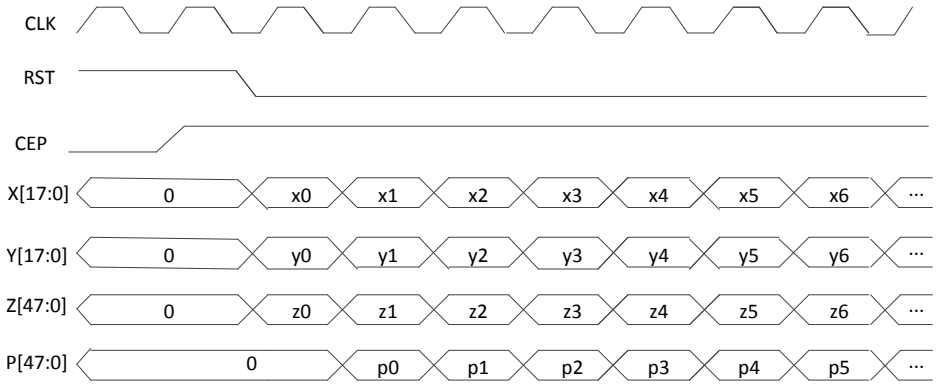



图 2-8 通用乘加模式的典型时序

### 2.3.3 乘累加模式

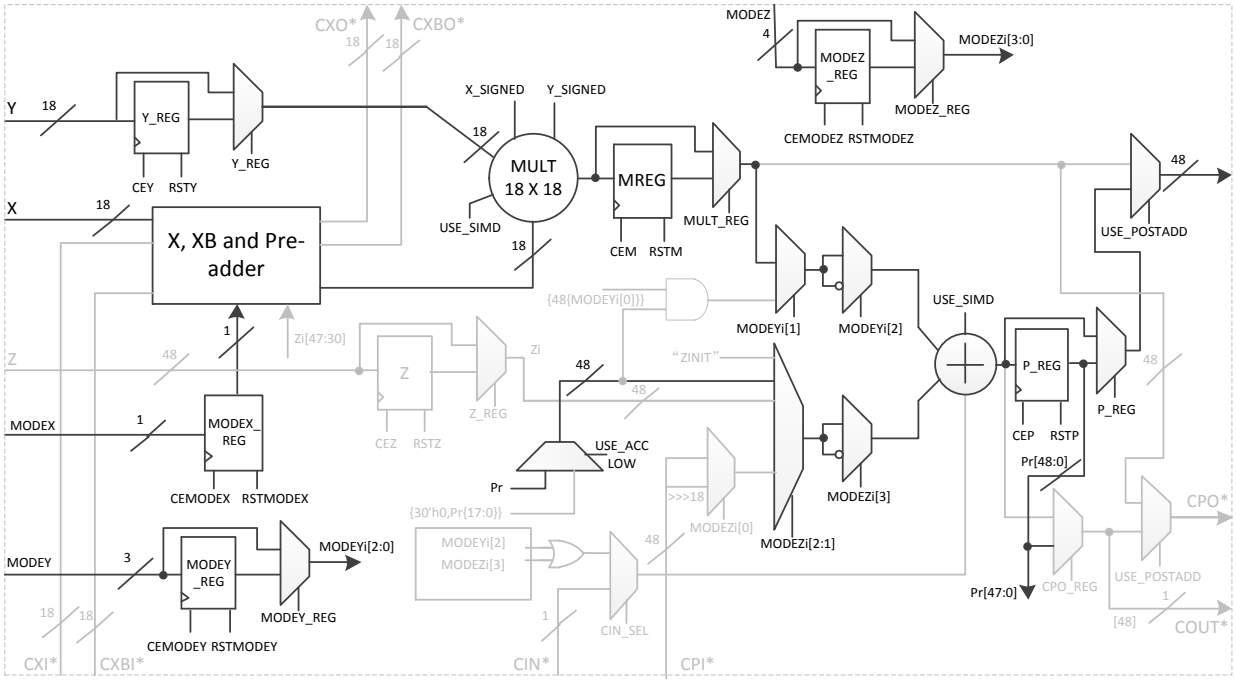



图 2-9 乘累加模式应用示意图

如上图所示，APM配置成乘累加模式时，其等效算术表达式为：

$$P=P \pm X \times Y$$

乘累加模式的主要特性有：

- 一个APM可实现一个18*18的乘累加运算(P为48bit)或两个9*9的乘累加运算(P为24bit)
- 支持有符号数和无符号数
- 可选的输入/输出寄存器
- P值可预置
- 乘累加功能需使能输出寄存器

使能 APM 中的 Preadd Unit 后，APM 可配置成带预加的乘累加模式，其算术表达式为：

$$P=P\pm Y\times(X\pm Z[47:30])$$

该模式下一个 APM 可实现一个(18±18)×18 的带预加的乘累加运算(P 为 48bit)或两个(9±9)×9 的带预加的乘累加运算(P 为 24bit)。带预加的乘累加模式有两级内部流水寄存器可供选择，其应用示意图如下图所示：

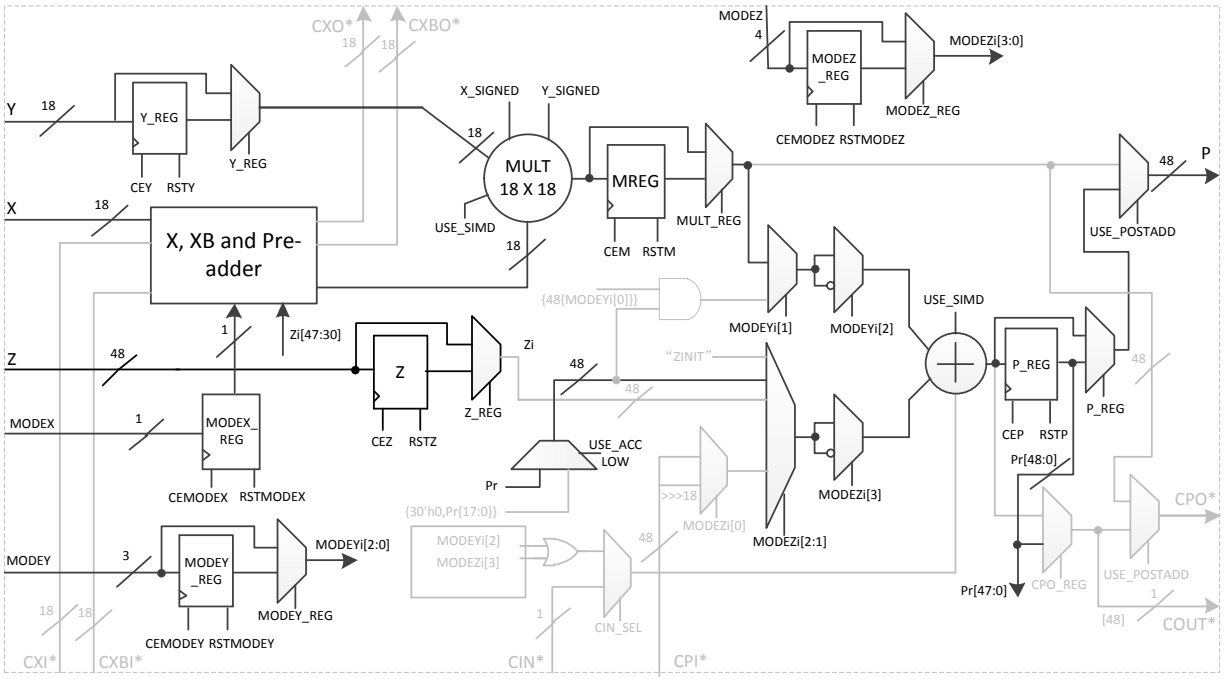



图 2-10 带预加的乘累加应用示意图

乘累加模式可以使用Logos Multiply-Accumulator IP进行例化（参见“Logos系列Module IP用户指南”之APM），也可以使用GTP_APM_E1单元进行例化。附录中包含使用GTP例化为乘累加模式的示例，供用户在使用时进行参考配置。

乘累加模式的典型时序如下，其中X=1，Y=1，P=P+X×Y，P在CEP有效时才能输出，使能P_REG后，P端对应X,Y端输入数据的计算结果在下一个时钟上升沿输出。

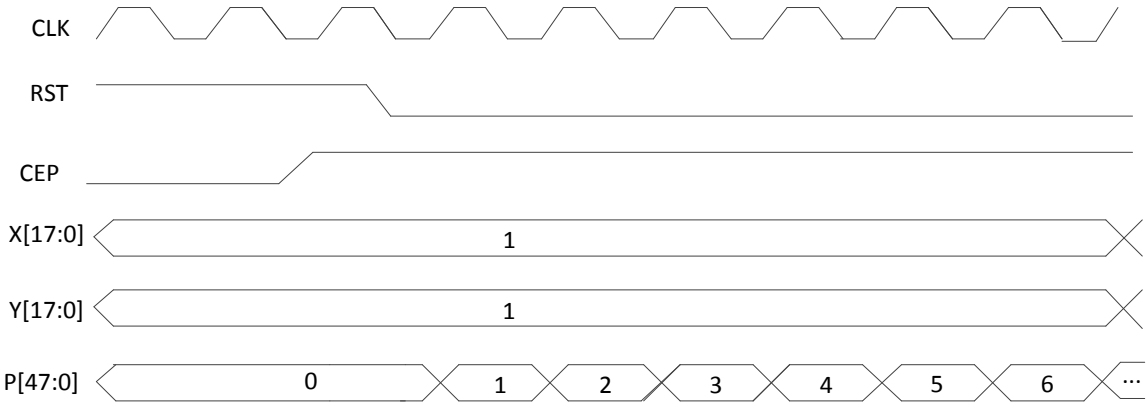



图 2-11 乘累加模式的典型时序

### 2.3.4 FIR 模式

通过APM级联也可实现FIR运算，典型的FIR filter可以描述为以下表达式：

$$y_n = \sum_{i=0}^{N-1} x_{n-i} h_i = x_n h_0 + x_{n-1} h_1 + \ldots + x_{n-N+1} h_{N-1}$$

x为输入的数据流，y为输出的数据流，h为系数。图 2-12为采用24个APM级联实现的24阶Systolic FIR运算的功能实现图。

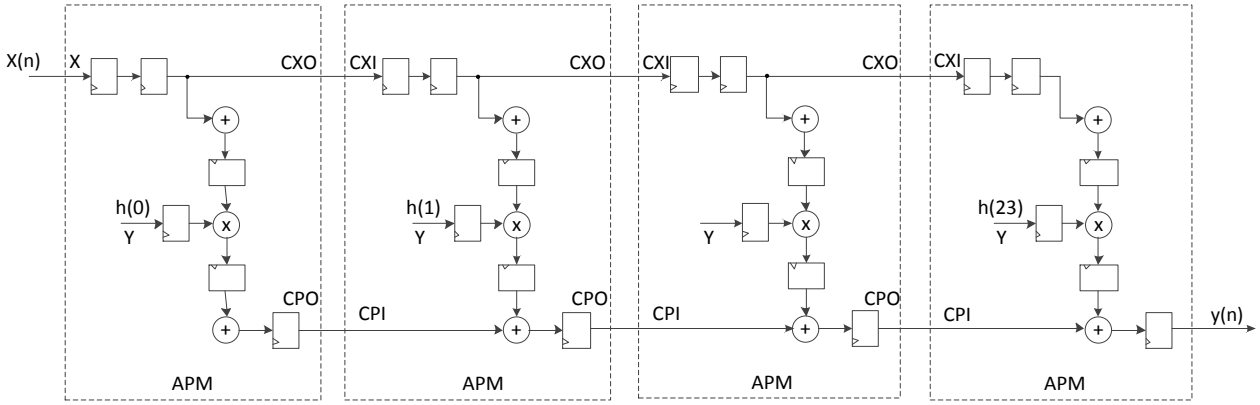



图 2-12 Systolic FIR 功能级联示意图

FIR模式需要通过对GTP_APM_E1单元进行级联后例化生成，通过对GTP_APM_E1单元的配置，可以实现多种形式的FIR运算，具体的实现类型及配置方法如下表所示（所有FIR配置里都有CPO_REG = 1）。

表 2-5 典型 FIR 滤波器实现

| 滤波器类型 | 第一级 APM | 中间级 APM | 最后一级 APM |
|---|---|---|---|
| Common setting: | CPO_REG=1 | | |
| Systolic FIR | CXO_REG=2<br>X_REG=1<br>X_SEL=0 | CXO_REG=2<br>X_REG=1<br>X_SEL=1 | X_REG=1<br>X_SEL=1 |
| 2-Channel Systolic FIR | CXO_REG=3<br>X_REG=1<br>X_SEL=0 | CXO_REG=3<br>X_REG=1<br>X_SEL=1 | X_REG=1<br>X_SEL=1 |
| Transposed FIR | CXO_REG=1<br>X_REG=1<br>X_SEL=0 | CXO_REG=0<br>X_REG=0<br>X_SEL=1 | X_REG=0<br>X_SEL=1 |
| Symmetrical FIR<br>(偶数 tap) | PREADD=1<br>CXO_REG=2<br>X_REG=1<br>X_SEL=0<br>XB_SEL=2'b01 | PREADD=1<br>CXO_REG=2<br>X_REG=1<br>X_SEL=1<br>XB_SEL=2'b01 | PREADD=1<br>CXO_REG=2<br>X_REG=1<br>X_SEL=1<br>XB_SEL=2'b11 |
| Symmetrical FIR<br>(奇数 tap) | PREADD=1<br>CXO_REG=2<br>X_REG=1 | PREADD=1<br>CXO_REG=2<br>X_REG=1 | PREADD=0<br>CXO_REG=1<br>X_REG=1 |

| | | | |
|---|---|---|---|
| | X_SEL=0<br>XB_SEL=2'b01 | X_SEL=1<br>XB_SEL=2'b01 | X_SEL=1<br>XB_SEL=2'b11 |
| 2-Channel Symmetrical FIR<br>（偶数 tap） | PREADD=1<br>CXO_REG=3<br>X_REG=1<br>X_SEL=0<br>XB_SEL=2'b10 | PREADD=1<br>CXO_REG=3<br>X_REG=1<br>X_SEL=1<br>XB_SEL=2'b10 | PREADD=1<br>CXO_REG=3<br>X_REG=1<br>X_SEL=1<br>XB_SEL=2'b11 |
| 2-Channel Symmetrical FIR<br>(奇数 tap) | PREADD=1<br>CXO_REG=3<br>X_REG=1<br>X_SEL=0<br>XB_SEL=2'b10 | PREADD=1<br>CXO_REG=3<br>X_REG=1<br>X_SEL=1<br>XB_SEL=2'b10 | PREADD=0<br>CXO_REG=1<br>X_REG=1<br>X_SEL=1<br>XB_SEL=2'b11 |

# 3 附录

## 3.1 级联可实现功能

通过对 APM 级联可以获得更高位宽运算，下表描述了 APM 级联可实现的功能，下表为举例，仅供参考。注意级联输入端口仅可连接上一级对应级联输出端口，级联输出端口仅可连接下一级对应级联输入端口，未使用时可悬空。

表 3-1 APM 级联可实现功能描述

| APM 级联 | 可实现功能 |
|---|---|
| 2 个 apm 级联 | 两个 apm 可实现 2 个（9×9+9×9）运算 |
| | 两个 apm 可实现 2 个[(9+9)×9+(9+9)×9]运算 |
| | 两个 apm 可实现 1 个输出位宽为 66 位的 P=P±18×18 运算 |
| | 两个 apm 可实现 1 个输出位宽为 96 位的 P=P±18×18 运算 |
| | 两个 apm 可实现 1 个（18×18+18×18）运算 |
| | 两个 apm 可实现 1 个(18+18)×18+(18+18)×18 运算 |
| | 两个 apm 可实现 1 个输出位宽为 66 位的 P=P±(18+18)×18 运算 |
| | 两个 apm 可实现 1 个输出位宽为 96 位的 P=P±(18+18)×18 运算 |
| | 两个 apm 可实现 1 个 18×36 运算 |
| 3 个 apm 级联 | 三个 apm 可实现 2 个输出位宽为 24 位的 P=P+(9×9+9×9)运算 |
| | 三个 apm 可实现 1 个输出位宽为 48 位的 P=P+(9×9+9×9)运算 |
| | 三个 apm 可实现 2 个输出位宽为 24 位的 P=P+((9+9)×9+(9+9)×9)运算 |
| | 三个 apm 可实现 1 个输出位宽为 48 位的 P=P+((9+9)×9+(9+9)×9)运算 |
| | 三个 apm 可实现 1 个输出为 48 位的 P=P+(18×18+18×18)运算 |
| | 三个 apm 可实现 1 个输出为 48 位的 P=P+((18+18)×18+(18+18)×18)运算 |
| | 三个 apm 可实现 1 个输出为 66 位的 P=P+18×36 运算 |
| | 三个 apm 可实现 1 个输出为 66 位的 P=P+(18+18)×36 运算 |
| 4 个 apm 级联 | 四个 apm 可实现 2 个[(9×9+9×9)+(9×9+9×9)]运算 |
| | 四个 apm 可实现 2 个[((9+9)×9+(9+9)×9)+((9+9)×9+(9+9)×9)]运算 |
| | 四个 apm 可实现 1 个输出为 66 位的 P=P+(18×18+18×18)运算 |
| | 四个 apm 可实现 1 个输出为 66 位的 P=P+((18+18)×18+(18+18)×18)运算 |
| | 四个 apm 可实现 1 个(18×18+18×18)+(18×18+18×18)运算 |
| | 四个 apm 可实现 1 个((18+18)×18+(18+18)×18)+((18+18)×18+(18+18)×18)运算 |
| | 四个 apm 可实现 1 个 27×27 运算 |
| | 四个 apm 可实现 1 个 36×36 运算 |
| | 四个 apm 可实现 1 个 36×18+36×18 运算 |
| | 四个 apm 可实现 1 个 36×(18+18)+36×(18+18)运算 |
| 6 个 apm 级联 | 六个 apm 可实现 1 个 P=P+27×27 运算 |
| 8 个 apm 级联 | 八个 apm 可实现 1 个 27×27+27×27 运算 |

注：各功能符号性具体见 2.1 中各模块操作数符号说明。

### 3.2 设计建议

- 进行高速滤波流水线应用时，建议使用 APM 级联功能，包括加法器级联。
- 乘法器由 MODEX、MODEY、MODEZ 等信号动态控制，具有很强的灵活性，设计中使用寄存器与动态配置模式，相较乘法器组合可更充分利用 APM 性能。
- 为实现 APM 的最高性能，需要使用全部流水寄存器，如果对延迟有要求，无法使用全部寄存器，请尽量使能 MREG。
- 使用内部级联端口而非通过 fabric routing 连接可以节省功耗。
- 进行较小位宽的运算时请进行符号位扩展。
- 多个 APM 进行级联时，不同信号通道的流水级数应匹配。

### 3.3 使用 GTP 例化为乘法模式

带预加的乘法器GTP例化模板如下：

- 通过USE_PREADD=1使能预加功能，不使能POSTADD
- 不使用X端口、Y端口等寄存器，使用MULT_REG
- MODEX=1'b0，PREADD模块为加法功能
- MODEY=3'b000，MODEY控制POSTADD功能，此处没有使能POSTADD
- MODEZ=4'b0000，Zmux选择为ZINIT，不对Zmux输出取负，此处没有使能POSTADD
- 输出结果P=(X+Z[47:30])×Y

```
GTP_APM_E1 #(
    .GRS_EN("TRUE"),
    .ASYNC_RST(1'b0),
    .X_SIGNED(1), //X 为有符号数
    .Y_SIGNED(1), //Y 为有符号数
    .X_REG(1'b0),
    .Y_REG(1'b0),
    .Z_REG(1'b0),
    .P_REG(1'b0),
    .CXO_REG(2'b00),
    .CPO_REG(1'b0),
    .MULT_REG(1),//使用 MULT_REG
    .PREADD_REG(1'b0),
    .MODEX_REG(1'b0),
    .MODEY_REG(1'b0),
    .MODEZ_REG(1'b0),
    .X_SEL(1'b0),
```

```
        .XB_SEL(2'b00),
        .CIN_SEL(1'b0),
        .USE_SIMD(1'b0),
        .USE_ACCLOW(1'b0),
        .USE_PREADD(1'b1), //使能预加功能
        .USE_POSTADD(1'b0),
        .Z_INIT(48'd0)
)
u0_GTP_APM_E1 (
        .CPO(),
        .CXBO(),
        .CXO(),
        .P(P),
        .CPI(),
        .CXBI(),
        .CXI(),
        .MODEY(3'b000),
        .MODEZ(4'b0000),
        .X(X),
        .Y(Y),
        .Z(Z),
        .COUT(),
        .CEM(1'b1),     //MULT_REG 时钟使能
        .CEMODEX(1'b0),
        .CEMODEY(1'b0),
        .CEMODEZ(1'b0),
        .CEP(0),
        .CEPRE(1'b0),
        .CEX(1'b0),
        .CEY(1'b0),
        .CEZ(1'b0),
        .CIN(),
        .CLK(Clk),
        .MODEX(1'b0),
        .RSTM(Rst),
        .RSTMODEX(Rst),
        .RSTMODEY(Rst),
        .RSTMODEZ(Rst),
        .RSTP(Rst),
        .RSTPRE(Rst),
        .RSTX(Rst),
        .RSTY(Rst),
        .RSTZ(Rst)
);
```

### 3.4 使用 GTP 例化为乘加模式

通用乘加的GTP例化模板如下：

- 通过USE_POSTADD=1使能累加功能，不使能PREADD功能
- 不使能X端口、Y端口等寄存器，使用P_REG
- MODEX=1'b0，PREADD模块为加法功能
- MODEY=3'b000，Ymux为乘法器输出，不对Ymux输出取负
- MODEZ=4'b0100，Zmux选择为Z输入端，不对Zmux输出取负
- 输出结果为P=X×Y+Z

```
GTP_APM_E1 #(
        .GRS_EN("TRUE"),
        .ASYNC_RST(1’b0),
        .X_SIGNED(1’b1), //X 为有符号数
        .Y_SIGNED(1’b1), //Y 为有符号数
        .X_REG(1’b0),
        .Y_REG(1’b0),
        .Z_REG(1’b0),
        .P_REG(1’b1), //使用 P_REG
        .CXO_REG(2’b00),
        .CPO_REG(1’b0),
        .MULT_REG(1’b0),
        .PREADD_REG(1’b0),
        .MODEX_REG(1’b0),
        .MODEY_REG(1’b0),
        .MODEZ_REG(1’b0),
        .X_SEL(1’b0),
        .XB_SEL(2’b00),
        .CIN_SEL(1’b0),
        .USE_SIMD(1’b0),
        .USE_ACCLOW(1’b0),
        .USE_PREADD(1’b0),
        .USE_POSTADD(1’b1), //使能累加功能
        .Z_INIT(48'd0)
)
u0_GTP_APM_E1 (
        .CPO(),
        .CXBO(),
        .CXO(),
        .P(P),
        .CPI(),
        .CXBI(),
```

```
    .CXI(),
    .MODEY(3'b000),
    .MODEZ(4'b0100),
    .X(X),
    .Y(Y),
    .Z(Z),
    .COUT(),
    .CEM(1'b0),
    .CEMODEX(1'b0),
    .CEMODEY(1'b0),
    .CEMODEZ(1'b0),
    .CEP(1'b1),        //P_REG 时钟使能
    .CEPRE(1'b0),
    .CEX(1'b0),
    .CEY(1'b0),
    .CEZ(1'b0),
    .CIN(),
    .CLK(Clk),
    .MODEX(1'b0),
    .RSTM(Rst),
    .RSTMODEX(Rst),
    .RSTMODEY(Rst),
    .RSTMODEZ(Rst),
    .RSTP(Rst),
    .RSTPRE(Rst),
    .RSTX(Rst),
    .RSTY(Rst),
    .RSTZ(Rst)
);
```

### 3.5 使用 GTP 例化为乘累加模式

乘累加的GTP例化模板如下：通过USE_POSTADD=1使能累加功能，不使能PREADD功能

- 不使能X端口、Y端口等寄存器，使用P_REG
- MODEX=1'b0，PREADD模块为加法功能
- MODEY=3'b000，Ymux为乘法器输出，不对Ymux输出取负（取负结果变为P=P-X×Y）
- MODEZ=4'b0010，Zmux选择为累加器反馈，不对Zmux输出取负（取负结果变为P=-P+X×Y）
- 在乘累加模式中，需注意先将MODEZ设为4’b0000，将APM内部的累加值初始为零，再将MODEZ值设为4’b0010进行乘累加计算

- 输出结果为P=P+X×Y

```
GTP_APM_E1 #(
        .GRS_EN("TRUE"),
        .ASYNC_RST(1’b0),
        .X_SIGNED(1’b1), //X 为有符号数
        .Y_SIGNED(1’b1), //Y 为有符号数
        .X_REG(1’b0),
        .Y_REG(1’b0),
        .Z_REG(1’b0),
        .P_REG(1’b1), //使用 P_REG
        .CXO_REG(2’b00),
        .CPO_REG(1’b0),
        .MULT_REG(1’b0),
        .PREADD_REG(1’b0),
        .MODEX_REG(1’b0),
        .MODEY_REG(1’b0),
        .MODEZ_REG(1’b0),
        .X_SEL(1’b0),
        .XB_SEL(2’b00),
        .CIN_SEL(1’b0),
        .USE_SIMD(1’b0),
        .USE_ACCLOW(1’b0),
        .USE_PREADD(1’b0),
        .USE_POSTADD(1’b1), //使能累加功能
        .Z_INIT(48'd0)
)
u0_GTP_APM_E1 (
        .CPO(),
        .CXBO(),
        .CXO(),
        .P(P),
        .CPI(),
        .CXBI(),
        .CXI(),
        .MODEY(3'b000),
        .MODEZ(MODEZ),
        .X(X),
        .Y(Y),
        .Z(Z),
        .COUT(),
        .CEM(1'b0),
        .CEMODEX(1'b0),
        .CEMODEY(1'b0),
        .CEMODEZ(1'b0),
```

```
    .CEP(1'b1),        //P_REG 时钟使能
    .CEPRE(1'b0),
    .CEX(1'b0),
    .CEY(1'b0),
    .CEZ(1'b0),
    .CIN(),
    .CLK(Clk),
    .MODEX(1'b0),
    .RSTM(Rst),
    .RSTMODEX(Rst),
    .RSTMODEY(Rst),
    .RSTMODEZ(Rst),
    .RSTP(Rst),
    .RSTPRE(Rst),
    .RSTX(Rst),
    .RSTY(Rst),
    .RSTZ(Rst)
);
```

# 免责声明

## 版权声明

本文档版权归深圳市紫光同创电子有限公司所有，并保留一切权利。未经书面许可，任何公司和个人不得将此文档中的任何部分公开、转载或以其他方式披露、散发给第三方。否则，公司必将追究其法律责任。

## 免责声明

1、本文档仅提供阶段性信息，所含内容可根据产品的实际情况随时更新，恕不另行通知。如因本文档使用不当造成的直接或间接损失，本公司不承担任何法律责任。

2、本文档按现状提供，不负任何担保责任，包括对适销性、适用于特定用途或非侵权性的任何担保，和任何提案、规格或样品在他处提到的任何担保。本文档在此未以禁止反言或其他方式授予任何知识产权使用许可，不管是明示许可还是暗示许可。

3、公司保留任何时候在不事先声明的情况下对公司系列产品相关文档的修改权利。